

AI_MarketReplay

FINAL 5★ DATA + REPLAY + SELF-AUDIT + CODE-CORRECTION SPECIFICATION

Continuous Engineering Intelligence Loop

1 1. Mission

AI_MarketReplay is the historical intelligence, diagnosis, repair, and validation brain of the trading system. It must not merely answer:

"Did the trade win or lose?"

It must answer:

"What did the strategy believe at the exact decision moment, what actually happened afterward, where did the strategy's thesis first diverge from reality, which component caused that divergence, why did that component fail, what part of its implementation is responsible, what correction should be made, and does the correction genuinely improve the strategy on unseen data?"

The complete loop is:

```
LIVE STRATEGY -> TRADE / SIGNAL -> FIREBASE -> ENCODER ->  
COMPRESSED STORAGE -> DECODER -> FULL RECONSTRUCTED ANALYSIS ->  
AI_MarketReplay -> REPLAY -> FIRST DIVERGENCE -> ROOT CAUSE ->  
FAILED COMPONENT -> FAILED LOGIC / PARAMETER / INTEGRATION ->  
SOURCE-CODE INVESTIGATION -> CORRECTION HYPOTHESIS ->  
PATCH CANDIDATE -> REPLAY -> ABLATION / A-B / STRESS ->  
WALK-FORWARD -> OUT-OF-SAMPLE -> ACCEPT / REJECT ->  
VERSIONED STRATEGY IMPROVEMENT
```

2 2. Architecture

Everything must live under:

```
1 ai/  
2 aireplay/  
3     controller.py  
4     config.py  
5     models.py  
6     data_engine.py  
7     replay_engine.py  
8     intelligence_engine.py  
9     experiment_engine.py
```

```

10     learning_engine.py
11     validation_engine.py
12     reporting_engine.py
13     cli.py
14
15     datasets/
16     experiments/
17     artifacts/
18     reports/
19     tests/

```

The architecture intentionally stays compact. The intelligence belongs in the core modules rather than being fragmented into dozens of tiny files.

3 3. Existing Firebase Data Model

The existing Firebase trade document is the historical source of truth. Conceptually: trades/trade_{ticket}

A trade contains:

- Identity
- Execution / opening state
- Analysis (M1, M5, H1)
- Price evolution[]
- Result / closing state
- Analysis at close

The existing encoder/decoder architecture is important. The data flow is: FULL ANALYSIS -> ENCODER -> COMPRESSED REPRESENTATION -> FIREBASE -> DECODER -> FULL ANALYSIS -> AI_MarketReplay

Therefore: AI_MarketReplay must consume the decoded canonical representation, not attempt to reason directly from compressed Firebase data.

4 4. Trade Identity

Every replay record must preserve: ticket, symbol, magic, comment, order_type, entry_timestamp

And AI_MarketReplay must add: replay_id, dataset_version, strategy_version, analysis_schema_version, replay_engine_version, source_document_version

This makes every experiment reproducible.

5 5. Execution Data

The opening execution state must preserve: entry_price, volume, stop_loss, take_profit, take_profit_2, take_profit_3, actual_margin, actual_risk_usd, risk_percent_used, spread_at_entry

magic, comment

This allows Replay to distinguish **prediction failure** from **execution failure**.

6 6. Opening Decision

The exact decision state must be reconstructed. It should include: `predicted_direction`, `final_decision`, `simple_action`, `confidence`, `probability_of_hit`, `risk_reward_ratio`, `reason`

Where available: `decision_probability`, `post_chain_probability`, `probability_ledger`, `directional_p`, `directional_veto`

The opening decision must be treated as **immutable historical evidence**.

7 7. DecisionGenome

The central structure is `DecisionGenome`. It represents everything the strategy knew and believed at T0. It should contain:

- **Market state:** price, bid, ask, spread, volume, ATR, volatility
- **Trend:** trend_bias, EMA (9, 20, 21, 50, 100, 200), ADX
- **Market structure:** HH, HL, LH, LL, BOS, CHOCH, swing_structure, microstructure
- **Liquidity:** liquidity_zones, liquidity_sweep, sweep_direction/strength/age
- **FVG:** FVG_exists, direction, size, age, fill_percentage, status, quality
- **VWAP:** VWAP, price_vs_VWAP, slope, distance, regime
- **RVAM:** relative_volume, volume_anomaly, volume_regime
- **CLV / Absorption:** absorption_detected, direction, strength
- **Momentum:** RSI, RSI_direction, RSI_divergence, MACD, stochastic, Bollinger
- **Supply / Demand:** zone_type, grade, quality, distance_to_zone
- **Context:** Wyckoff/ICT state, Session (phase), Regime (market, trend, volatility, liquidity), News (state, event_risk)
- **Entry:** entry_quality, zone, distance_from_structure

8 8. Gate State

Every decision gate must be preserved. Not simply `VWAP_GATE = FAIL`, but: `gate_name`, `value`, `threshold`, `margin`, `near_miss`, `enforced`, `passed`, `failed`

This allows Replay to discover whether thresholds are intelligently calibrated.

9 9. Probability Ledger

The system must preserve how the final probability was constructed. Conceptually: `initial probability + trend + structure + liquidity + FVG + VWAP + RVAM + absorption + momentum + microstructure + gate adjustments + veto adjustments = final probability`

This is essential for diagnosing overweighted components. If the prediction fails, Replay can investigate whether a specific contribution (e.g., FVG +12%) was unjustified rather than blaming the entire strategy.

10 10. Indicator Recommendation State

Each component should retain its own recommendation: `component, direction, recommendation, confidence, strength, quality, available`

This allows component-level evaluation.

11 11. Price Evolution

The existing `price_evolution[]` is the market's response to the decision. Each observation may contain: `timestamp, price, profit_usd, profit_percent, distance_from_entry_pips, spread, volume, analysis_m1/m5/h1`

The Replay architecture should support finer data later, including candle-level or tick/quote-level evolution where available.

12 12. Temporal Market Reality

For every evolution point, a `MarketReality` must be generated. Example: `timestamp, price, return_since_decision, MFE, MAE, distance_to_entry/SL/TP, time_since_decision, spread(_change), volume(_change), volatility(_change)`

Where reconstructed analysis permits: changes in structure, trend, liquidity, FVG, VWAP, RVAM, absorption, microstructure, and regime.

13 13. Future Outcome

Future information is used **only for evaluation**. For each decision (+1m, +5m, +1h, etc.), calculate: `future_return, MFE, MAE, direction, SL_hit, TP1_hit, time_to_SL/TP`

This makes it possible to distinguish wrong direction from right direction but bad timing, or bad entry, or unrealistic target.

14 14. Replay Engine

The Replay Engine freezes the strategy's decision at T_0 . Then walks forward: $T_0 \rightarrow T+1 \rightarrow \dots \rightarrow T_n$. At every point it compares: **STRATEGY BELIEF AT T_0 vs MARKET REALITY AT $T+n$** . The historical decision must never be contaminated by future information.

15 15. First Divergence

This is one of the most important outputs. The diagnosis becomes **FIRST DIVERGENCE = $T+3$** , not simply **TRADE = LOSS**.

16 16. Failure Classification

Every failure should be categorized (e.g., prediction_error, timing_error, execution_error, structure_error, liquidity_error, gate_error).

17 17. Root Cause Hierarchy

The system should produce: PRIMARY CAUSE, SECONDARY CAUSE, CONTRIBUTING CAUSES, PROTECTIVE COMPONENTS, IRRELEVANT COMPONENTS

18 18. Component Failure Is Not the End

The system must never stop at "SMC failed." It must continue: SMC failed $\rightarrow$ Which subsystem? (BOS) $\rightarrow$ Why? (False BOS) $\rightarrow$ Why false? (Liquidity sweep classified as break) $\rightarrow$ Why did implementation allow that? $\rightarrow$ Inspect algorithm $\rightarrow$ Identify responsible function $\rightarrow$ Identify parameter $\rightarrow$ Generate correction

19 19. Source-Code Intelligence

The final system must have a strategy implementation map: COMPONENT $\rightarrow$ FILE $\rightarrow$ CLASS $\rightarrow$ FUNCTION $\rightarrow$ INPUTS $\rightarrow$ PARAMETERS $\rightarrow$ OUTPUT $\rightarrow$ DECISION CONTRIBUTION

20 20. Algorithm Failure vs Parameter Failure

The AI must distinguish them:

- **Algorithm failure:** Logic is flawed (e.g., BOS = price crossed swing, ignoring liquidity sweeps).
- **Parameter failure:** Logic is sound, but parameter (e.g., SWING_LENGTH = 5) fails in a specific regime.
- **Data, Timing, Integration, Weighting, Regime failures.**

21 21. SMC Example

Instead of just knowing SMC fails 39% of the time, Replay decomposes failures (e.g., BOS false positives 42%) and analyzes them by regime (e.g., high volatility, liquidity sweeps). Now the AI has discovered *how* SMC fails.

22 22. Correction Hypothesis

The AI formulates a hypothesis based on evidence. e.g., "A valid BOS should require confirmed structural break, displacement, and rejection of immediate sweep patterns." This remains a hypothesis, not an accepted change.

23 23. Source-Code Correction

The system should produce a structured proposed change: `CorrectionProposal`. Where feasible, generate a patch candidate. However, **AI_MarketReplay must never automatically modify production strategy code simply because the historical result looks better.** The correction must pass validation.

24 24. Code-Level Diagnosis

Aim to produce specific mappings (e.g., File: `smc_engine.py`, Function: `detect_bos()`) based on the actual source repository, not invented names.

25 25. Code Repair Loop

Failure -> Diagnosis -> Root cause -> Source location -> Correction hypothesis -> Candidate patch -> Unit tests -> Historical replay -> Ablation -> A/B -> Stress test -> Walk-forward -> OOS -> Accept / reject

26 26. Unit-Level Validation

Before strategy-level validation, the candidate correction must be tested against the specific failure and historically valid cases to prevent over-correction.

27 27. Avoiding the Fake Fix

A component that becomes accurate by refusing to generate signals is not necessarily better. The AI must evaluate trade frequency, expectancy, profit factor, drawdown, etc.

28 28. Ablation Testing

Test BASELINE against combinations (e.g., WITHOUT_SMC, WITHOUT_FVG) to answer: "Does this component actually add value?"

29 29. A/B Testing

Compare current vs. corrected components across accuracy, expectancy, profit factor, MFE/-MAE, and regimes (trend, range, volatility).

30 30. Interaction Intelligence

Analyze combinations (e.g., FVG + Liquidity) and confluence chains to discover which combinations produce reliable decisions under which conditions.

31 31. Microstructure

Microstructure should remain the final trigger, not the main market predictor. The decision chain conceptually remains: Macro context -> ... -> Confluence -> Microstructure -> Entry.

32 32. Gate Analysis

For every gate, calculate correct/incorrect rejection/acceptance rates to reveal if a gate is too strict or too weak.

33 33. Parameter Intelligence

Parameters must be treated as hypotheses. Replay should discover whether performance depends on regime (e.g., `SWING_LENGTH=5` works in low volatility but fails in high). The solution might be regime-adaptive logic.

34 34. Regime-Aware Repair

Evaluate corrections against regimes. A global correction is preferred only when evidence supports it. Otherwise, use REGIME-CONDITIONAL LOGIC.

35 35. Learning Engine

Prioritize structured/tabular models (Logistic Regression, LightGBM) to estimate probability, reliability, expected MFE/MAE. Deep temporal models should only be introduced if justified. Generative models are not the first solution.

36 36. Why Diffusion Is Not the First Step

First maximize information from existing data. Only later consider market simulation or synthetic paths. Synthetic data must never silently substitute real out-of-sample evidence.

37 37. Leakage Firewall

Non-negotiable: At T_0 , only use information available at or before T_0 . Future info is only for evaluation. Replay must enforce: `available_at <= decision_timestamp`.

38 38. Immutable Historical Data

AI_MarketReplay must never rewrite historical analysis. Use versioned extraction to create canonical datasets and experiments.

39 39. Experiment Engine

Every proposed improvement becomes an experiment with a baseline, candidate, hypothesis, and tracked metrics.

40 40. Validation Engine

No candidate becomes production-ready without: Historical replay, Ablation, A/B, Stress, Walk-forward, and Out-of-sample validation.

41 41. Promotion Gate

A correction receives: REJECTED, PROMISING, VALIDATED, PRODUCTION_CANDIDATE. The AI must be allowed to conclude: *"No change should be made."*

42 42. Reporting

Every replay should produce a human-readable report detailing the decision, outcome, divergence, root cause, candidate correction, and validation results.

43 43-44. Component & Strategy Health Reports

Continuously monitor accuracy, failure rates, main failures, worst regimes, and suggested actions for both individual components and the overall strategy.

44 45. What AI_MarketReplay Must Ultimately Say

It must provide a comprehensive narrative of what was believed, what happened, where it diverged, the root cause, implementation flaw, proposed fix, and validation status, rather than just "The trade lost."

45 46. The Same Intelligence Applies to Every Component

The exact same pipeline (Observe -> ... -> Validate -> Version) must exist for every component (SMC, FVG, VWAP, Gates, Risk Engine, etc.).

46 47-56. Core Engines & CLI

The architecture is divided into specific engines (Controller, Models, Data, Replay, Intelligence, Experiment, Learning, Validation, Reporting) accessible via a CLI.

47 57. AI Is an Auditor Before It Is a Controller

Initially, it observes, diagnoses, and proposes. It should not automatically control live trading until validated.

48 58. What Makes This 5★

It answers: *"What did we believe, what actually happened, exactly where were we wrong, which component caused it, why did its implementation produce that error, what change could fix it, and is that change demonstrably better without overfitting?"*

49 59. Final Intelligence Loop

The final system is an engineering intelligence loop that continuously discovers where the trading system is wrong, traces the error to its source, proposes a concrete repair, and attempts to prove that repair wrong before allowing it to become an improvement.